

检索什么才能有效地检索增强代码生成？实证研究及以后

古文超

德国慕尼黑海爾布隆工业大学

wenchao.gu@tum.de

陈板

中山大学，珠海，中国

chenjt75@mail2.sysu.edu.cn

王彦林

中山大学，珠海，中国

yanlin-wang@outlook.com

Tianyue Jiang

中山大学，珠海，中国

jiangty9@mail2.sysu.edu.cn

Xingzhe Li

中山大学，珠海，中国

lixzh75@mail2.sysu.edu.cn

Mingwei Liu

中山大学，珠海，中国

liumw26@mail.sysu.edu.cn

刘西林

华为云计算技术有限公司，中国深圳
liuxilin3@huawei.com

尉迟马

华为云计算技术有限公司，中国深圳
mayuchi1@huawei.com

Zibin Zheng

中山大学，珠海，中国
zhzibin@mail.sysu.edu.cn

摘要

由于复杂的代码依赖关系和大型语言模型（LLM）在处理长上下文方面的局限性，存储库级代码生成仍然具有挑战性。虽然检索增强生成（RAG）框架被广泛采用，但不同检索到的信息源——上下文代码、API 和类似片段——的有效性尚未得到严格分析。通过对两个基准测试的实证研究，我们证明上下文代码和潜在 API 信息显着提高了 LLM 性能，而检索到的类似代码通常会引入噪声，导致结果降级高达 15%。基于初步结果，我们提出了 AllianceCoder，这是一种新颖的上下文集成方法，它采用思想链提示将用户查询分解为实现步骤，并通过语义描述匹配检索 API。通过对 CoderEval 和 RepoExec 的广泛实验，AllianceCoder 实现了最先进的性能，比现有方法提高了高达 20% 的 Pass@1。这项研究提供了一个实验框架，以进一步探索在基于 RAG 的代码生成中检索什么，我们的复制包可在 <https://anonymous.4open.science/r/AllianceCoder>，以促进未来的研究。

CCS 概念

· 软件及其工程自动编程；· 信息系统信息检索查询处理；· 计算方法自然语言处理。

关键词

代码生成、法学硕士、实证研究

ACM 参考格式：

古文超、陈俊涛、王彦林、蒋天月、李兴哲、刘明伟、刘西林、马宇驰和郑子彬。2018。为了有效检索 - 增强代码生成，检索什么？实证研究及以后。在确保从您的权利确认电子邮件（会议首字母缩略词 'XX'）中输入正确的会议标题。ACM，纽约，纽约，美国，12 页。<https://doi.org/XXXXXXX.XXXXXXX>

1 介绍

在现代软件开发中，代码生成 [27,43,44,60] 已成为弥合开发人员的自然语言需求和执行代码之间差距的关键能力。随着大型语言模型（LLM）的出现，最先进的模型已经展示了从自然语言查询生成独立代码片段的令人印象深刻的能力。然而，这些成就主要限于功能级代码生成。存储库级代码生成——生成具有存储库上下文的代码的任务——由于存储库中代码依赖关系的复杂性以及具有长上下文的 LLM 的固有限制，仍然是一项具有挑战性的任务。为了应对这一挑战，大多数现有方法 [11,31,49,50,57] 采用检索增值生成（RAG）框架，该框架从存储库中检索相关信息，将其附加到查询中，然后将其输入 LLM 以进行代码生成。现有作品在检索阶段采用了不同的信息源。（1）

RepoForm [50]、RepoMinCoder [28] 和 R^2C^2 -Coder [11] 利用来自当前文件的上下文信息来增强代码生成。通过

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

arXiv:2503.20589v1 [cs.SE] 2025年3月26日

结合相关的本地上下文和对第三方库的引用，这些方法使 LLM 能够更有效地推断和完成目标函数。

(2) RepoFuse [29] 通过分析 import 语句提取相关 API，而 A3CodeGen [31] 首先生成伪代码。尽管这些信息源广泛用于基于 RAG 的存储库级代码生成，但它们对 LLM 性能的影响尚未得到深入研究。为了解决这个问题，我们进行了一项初步研究，分析不同类型的检索信息如何有助于提高存储库级代码生成中的 LLM 性能。我们的实验表明，有趣的是，与其在存储库级代码生成中的有效性相反，检索相似的代码片段并不总是提高存储库级代码生成中的 LLM 性能。在某些情况下，基于这些发现，我们得出结论，从当前文件和相关 API 中检索上下文信息对于提高 LLM 在存储库级代码生成中的性能至关重要。因此，为了解决这个问题，我们提出了一种简单而有效的方法。在存储库 API 处理阶段，我们利用 LLM 为存储库中的所有 API 生成自然语言描述。然后将这些描述编码为嵌入向量，以进行 API 检索。在查询处理阶段，我们用示例引导 LLM 将用户查询分解为多个详细的实现步骤。对于每个步骤，模型生成可能被调用的潜在 API 的描述。然后，生成的 API 描述序列由 LLM 细化并编码为表示

使用与存储库 API 处理阶段相同的在上下文集成代码生成阶段，我们利用这些 API 表示向量基于余弦相似度在存储库中检索最相关的 API。我们在两个公共数据集上评估我们的方法，CoderEval 和 RepoExec，我们的贡献可归纳如下：并详细解释我们进行了一项实证研究，以调查不同类型的检索信息如何影响存储库级代码生成中的 LLM 性能。

我们提出了一种简单而有效的方法，名为 AllianceCoder，它详细描述了检索增强生成（RAG）框架。我们进行了广泛的实验来评估我们的方法。结果表明，我们的方法优于先前的检索方法。这些描述基于

2 背景

2.1 检索增强生成

检索增强生成（RAG）通过集成外部知识来增强生成模型 [25]。传统模型虽然连贯，但由于依赖于固定记忆 [49,57]，经常遭受事实不准。RAG 框架包括一个检索器，它根据查询对相关信息进行排名，以及一个生成器，它根据查询和检索到的 RAG 已成功应用于需要事实准确性和知识基础的领域，包括问答 [2,54]、科学文本生成 [14] 和自动 RAG 过程可以制定如下：

$$A = \text{Generate}(q, \text{Retrieval}(q,$$

其中 q 表示用户查询， $c_1, \dots, c_n$

表示候选知识集， A 为语言模型生成的输出，函数 $\text{Retrieval}(\cdot)$ 基于给定查询从知识库中检索相关候

表 1: 不同基于 LLM 的方法检索的信息类型。

Approach	Similar Code	API	Context
A ³ CodeGen	✗	✓	✓
RepoCoder	✓	✗	✓
RepoFormer	✓	✗	✓
RepoMinCoder	✓	✗	✓
RLcoder	✓	✗	✓
R ² C ² – Coder	✓	✗	✓
GraphCoder	✓	✗	✓
RepoFuse	✓	✓	✓

将检索到的信息与原始查询相结合存储库级代码生成特别适合 RAG 框架。软件存储库包含大量的历史代码、库函数、API 调用和上下文信息，因此仅依靠用户查询来生成代码是不切实际的。此外，由于 LLM 的最大输入长度限制，将整个存储库导入模型是不可行的。RAG 通过从存储库中检索相关代码片段

2.2 存储库级代码生成

在 LLM 出现之前，由于深度学习模型的性能受限和输入长度短，代码生成方面的大多数研究工作仅限于独立的代码片段。然而，随着在这种情况下，RAG 框架已被广泛采用于基于 LLM 的存储库级代码生成方法中。存储库级代码生成过程可以制定如下：

$Code = Generate(q, Retrieval$

表 1 总结了当前基于 LLM 的存储库级代码生成方法中使用的检索信息相似代码是指从同一存储库中检索到的相似代码片段。基本假设是相似代码 $Codebase$ 中提取相关信息，同时 $Generate(\cdot)$ 将检索到的信息与原始查询进行集成，并利用 LLM 生

与目标代码共享某些特征，为 LLM 提供有用的实现提示。从表 1 API 检索侧重于识别应该在目标代码中调用的相关 API。目标是减少代码冗余，同时启用定制和支持复杂功能。有关可用 API 的信息有助于工程并减少目标实现中的部分上下文是指来自与用户查询相同文件的上下文信息。它包括库导入、类定义和成员函数实现等详细信息。这些信息有助于 LLM 更好地理解代码风格，但这一做法也存在限制。尽管当前基于 LLM 的代码生成方法广泛检索此类信息，但每种方法检索的信息类型和集成方式有所不同。在本节中，我们介绍了实验中正在使用的数据集、基线和 LLM 以及我们的实验是在 CoderEval [53] 和 RepoExec [20] 上进行的。CoderEval 是一个基准，用于评估实用代码生成任务上的代码生成性能。它由 230 个 Python 和 230 个来自真实世界开源项目的 Java 任务组成。每个任务都包含一个函数签名、一个函数描述

2.2 评估基线和 LLM

我们探索了两种最先进的存储库级代码生成框架：RepoCoder [57] 和 RLCoder [49]。RepoCoder 是一种迭代 retrieval-generation 管道。它有效地利用分散在存储库内不同文件上下文的信息，并可以生成各种颗粒度的代码的能力。每个任务都提供了开发人员指定的基本代码依赖关系，以及用于验证功能正确性的测试

何时检索以及保留哪些候选。这两个框架都证明了在存储库级代码生成

3.3 嵌入模型 方法有显著改进

UniXcoder [16] 是一个密集检索器，将查询和代码片段编码为密集矢量表示。这种向量化机制能够根据各向量嵌入之间的计算相似

3.4 指标

由于多个实现可以实现相同的功能，我们采用 $Pass@k$ [8] 作为我们唯一的评估指标，在我们的评估

3.5 实施细节 只有通过所有

在我们所有的实验中，我们使用了 GPT-4o Mini 和双子座 1.5 闪存，默认温度设置为 0.7。在相似代码检索的初步研究中，我们选择了前

4 初步研究 可以改进生成代码

为了研究不同信息类型对 LLM 在 RAG 框架下生成存储库级代码能力的影响，我们进行了一项初步研究。在本研究中，我们检查了三种类型上下文信息：此信息类型包括同一文件中目标函数之前的所有文本，例如库导入语句、类定义和其他已实现的函数。通过提供此类相关代码：在以前基于模型的方法的基础上，我们利用 LLM 将存储库中的所有候选代码编码为向量表示。假设事先知道正确的目标调用的 API：我们假设在目标代码中调用的所有 API 的先验知识利用上下文信息根据向量相似性检索前五个最相关的代码片段。该设置的实验结果表示为相似 LLM

表 2：不同检索信息类型的 LLM 存储库级代码生成性能的初步实验结果（以粗体显示的最佳性能）

Method	RepoExec			CoderEval		
	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
PureGPT	16.62	19.72	23.66	18.26	25.22	26.52
ContextGPT	29.01	34.08	38.59	30.43	36.52	37.83
SimilarGPT	16.90	23.38	24.79	20.00	23.04	23.48
APIGPT	25.63	32.39	34.93	25.65	30.43	33.04
ConSimGPT	22.53	26.48	27.60	14.35	20.43	23.04
ConAPIGPT	37.75	44.51	47.04	36.52	41.30	41.47
SimAPIGPT	25.36	31.55	34.09	22.17	26.52	26.52
ConSimAPIGPT	29.86	38.31	38.87	19.57	23.91	26.52
PureGemini	16.90	20.28	23.10	17.39	21.74	25.22
ContextGemini	30.99	34.08	35.77	23.04	26.09	27.39
SimilarGemini	17.18	22.25	23.38	13.48	16.52	16.96
APIGemini	26.20	30.99	34.93	22.61	24.78	26.09
ConSimGemini	24.22	27.33	29.02	13.91	15.22	15.22
ConAPIGemini	38.59	41.69	44.51	25.22	27.39	28.70
SimAPIGemini	27.32	31.27	32.96	19.13	22.17	22.61
ConSimAPIGemini	32.67	36.90	38.59	22.17	23.04	23.91

在我们的实验中，我们评估了三种类型检索信息的所有可能组合。具体来说，ConSimLLM 表示当提供上下文信息和相关代码作为输入时 LLM 的性能，而ConAPIILM对为了评估 LLM 在存储库级代码生成中的内在能力，我们还进行了一总体而言，我们的初步研究评估了八种不同的设置，每种设置对应于相关代码和上下文信息的一种组合。

5 初步研究结果 检索的结果

在本节中，我们旨在回答以下研究问题。RQ1：每种类型的信息如何有用？RQ2：能否通过简单地集成上下文信息和调用的 API 来改进？RQ3：上下文信息和调用的 API 的性能改进是否相互独立？

5.1 RQ1: 每种类型的信息如何有助于存储库级代码生成的性能改

表 3 展示了我们初步研究的实验结果。如表所示，上下文信息为 LLM 在存储库级代码生成方面提供了最显著的性能改进。这一结果是意料之外的，因为上下文信息提供了丰富的背景知识。此外，我们观察到合并调用的 API 也大大提高了性能。这种改进源于两个关键因素。首先，在存储库中明确指定相关的 API 允许 LLM 直接访问所需的功能。此外，上下文信息和调用的 API 信息的结合产生了最佳的整体性能。从人类开发人员的角度来看，这些元素共同提供了存储库级代码生成所需的所有基本知识，从而能够更有效地实现目标功能。同样，LLM 可以利用这些先验知识来增强其生成代码的能力。然而，一个意想不到的发现是，检索到的相关代码对性能改进的贡献很小，甚至在与其它类型的补充信息结合时会降低性能。例如，在这两个数据集中，包含所有三种类型信息（上下文信息、调用的 API 和检索到的代码）的模型表现最差。这种退化很可能是由于检索到的相似代码和目标代码之间的差异。理想情况下，检索到的相关代码应该表现出与目标实现的功能相似性，作为指导 LLM 生成正确代码的参考。但是，如果检索到的代码与目标代码存在显著差异，这可能会导致 LLM 生成错误的代码。因此，在存储库中找到相似模式的代码并不总是有助于提高性能。

很难从存储库中检索功能相似的代码。在实践中，检索到的代码片段通常无法满足这一要求，而是提供了与目标功能无关的代码。

发现 1: 上下文信息和调用的 API 对性能改进有很大贡献。然而，包含类似的代码并不总是有助于提高性能。

5.2 RQ2: 上下文信息和调用的 API 的性能提升能否通过简单地集

为了进一步研究上下文信息与调用 API 相结合的有效性，我们分析了为了减轻这种随机性对我们实验结果的影响，我们使用 Pass@5 度量从图中，我们观察到

$ConAPI_{LLM}$ 成功地覆盖了

然而，我们也注意到某些测试用例通过了 ContextLLM 或 API_{LLM} 但 $ConAPI_{LLM}$ 失败。一种可能的解释是，图 1b、图 1c 和图 1d 揭示了大量通过 API_{LLM} 但 $ConAPI_{LLM}$ 失败的测试用例。为了进一步研究这个问题，我们分析了 API_{LLM} 和 $ConAPI_{LLM}$ 一次通过的测试用例以及仅

由于上下文信息自然连接到用户查询，我们通过首先提供相关的应用编程接口信息来构建提示，然后是上下文信息和用户查询。这种排序与应用程序编程接口相关的内容在提示中出现在离用户查询更远的地方。通过测试用例与 API 处理输入，但如果输入长度过长，提示

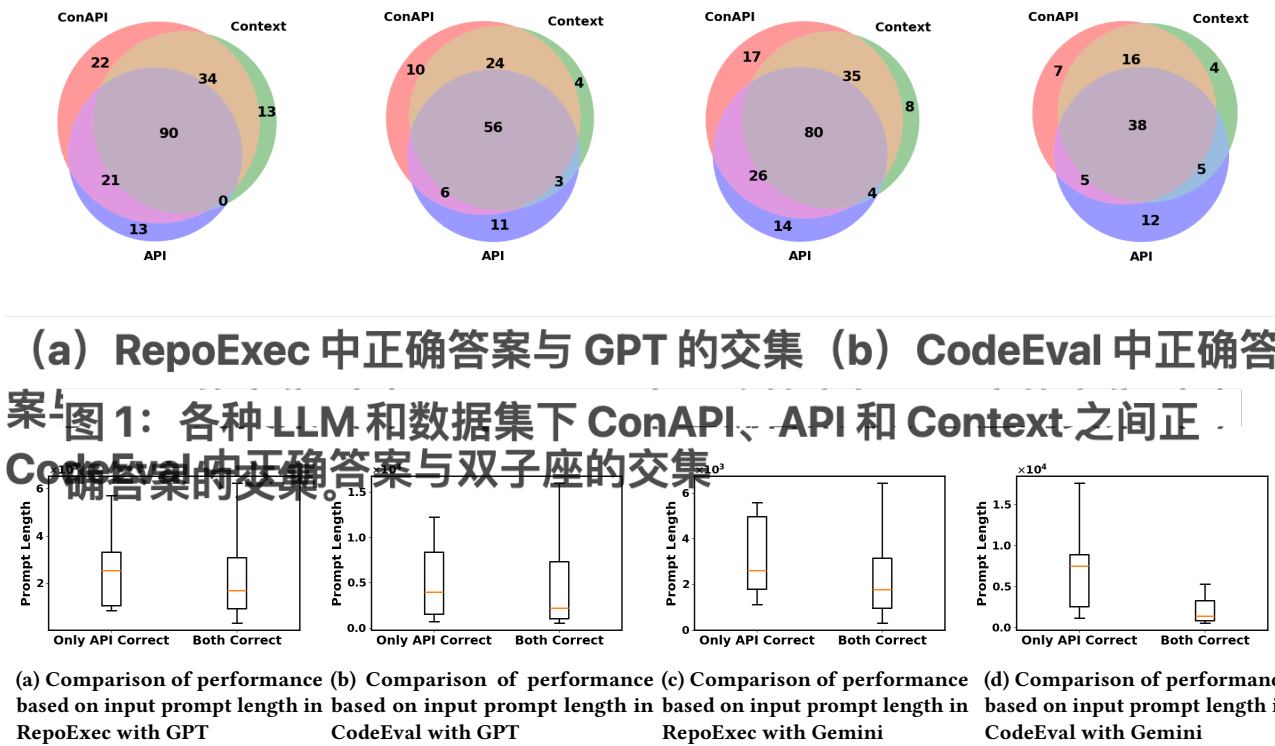


图 2: 测试用例的输入提示长度比较: ConAPI 和 API 的成功与跨不同数据集和 LLM 的仅 API 成功。

发现 2: 天真地附加带有上下文信息的调用 API 可能会导致

5.3 RQ3: 上下文信息和调用的 API 的性能改进是否相互独立?

由于上下文信息包含许多成员函数, 其中一些可能是被调用的函数, 因此上下文信息可能已经包含 API 详细信息。为了更好地理解不同上下文信息如何影响性能, 我们在此表中, 完全包含是指上下文信息包含所有调用的 API 详细信息的情况, 而不包括表示上下文信息不包含任何调用的 API 的情况。度量是完全包含被调用的 API 时, 以及 CPass 表示 ContextLLM 传递的测试用例数量, 而 BPass 表示 API 传递的测试用例数量。统计结果如表 3 所示。Context、LM 和 API_{LLM} 传递的

BPass 下的百分比表示 Context, M 通过的测试用例中 API_{LLM} 也成功通过的测试用例的比例。为从表 3 中, 我们观察到上下文数据中的 API 信息显着提高了性能。具体来说, 通过比较上下文信息完全此外, ContextLLM 和 API_{LLM} 传递的测试用例表现出很强的相似性。值得注意的是, 当上下文信息完全包含被调用的 API 此外, 我们承认上下文信息仍然提供了超越被调用 API 的有价值的内容。这测试用例也由 API_{LLM} 传递。测试用例成功这一被调用 API 明显看出, 即使它不包含任何被调用的 API 信息, 而仅通过上下文信息

表 3：通过测试用例的统计数据：仅通过上下文与不同 API 包含条件下使用 API 的上下文进行比较

LLM	RepoEval				CodeEval			
	Fully Contained		Not Included		Fully Contained		Not Included	
	CPass	BPass	CPass	BPass	CPass	BPass	CPass	BPass
GPT	30 (28.0%)	18 (60.0%)	36 (24.5%)	12 (33.3%)	8 (47.1%)	5 (62.5%)	29 (20.3%)	8 (27.6%)
Gemini	22 (20.6%)	15 (68.2%)	25 (16.7%)	8 (32.0%)	4 (22.2%)	2 (50.0%)	15 (10.3%)	2 (13.3%)

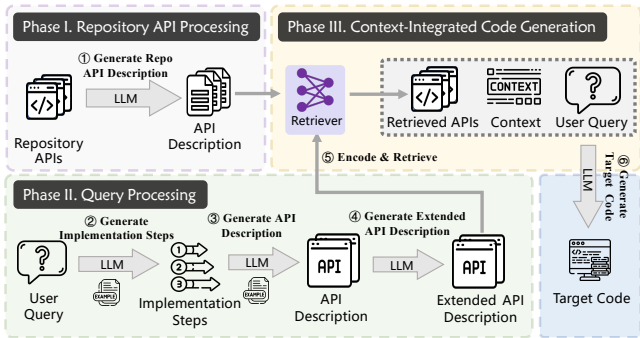


图 3：AllianceCoder 框架。

仅 API 信息不足以生成正确的代码。这表明上下文信息为代码生成发现 3：上下文信息有时包括目标代码的调用 API，而归因于上下文信息的部分性能改进实

6.1 概述 这些调用 API 的存储库根据我们的实证研究，潜在的 API 检索在存储库级代码生成中起着至关重要的作用，然而确定哪些 API 可以在存储库中调用仍然是一个具有挑战性的问题。为了解决这个问题，我们提出了 AllianceCoder，这是一种用于检索相关 API 的简单而有效的方法。如图 3 所示，AllianceCoder 由三个阶段组成：存储库 API 处理、查询处理和上下文集成代码生成。在存储库 API 处理阶段，我们利用 LLM 为存储库中

6.2 存储库 API 处理 6.2.1 生成 Repo API 描述。在我们提出的方法中，API 检索使用向量余弦相似度执行，编码过程由预训练模型以无监督方式处理。然而，编程语言和自然语言之间存在语义差距，这可能导致代码片段及其相应描述之间的余弦相似度较低。为了缓解这个问题，我们首先利用一旦生成了所有 API 的描述，它们就会被编码为表示向量并预先存储

6.2 查询处理 6.3.1 生成实现步骤。查询处理阶段的目标是生成可能在目标函数中调用的 API 的描述。为了实现这一点，我们首先提供示例并指示 LLM 将整个代码任务分解为一系列具体的实现步骤。这个过程可以制定如下：

6.3.2 生成 API 描述。一旦获得这些步骤，它们就会连同其他示例和提示一起反馈到 LLM 中，以生成每个步骤中使用的潜在 API 的描述。此过程表示如下：

功能的用户查询， $\{a_1, \dots, a_n\} = \text{Generate}(\{e_1, \dots, e_n\})$ 表示为上下文学习提供的示例，而其中 $s_1, \dots, s_n$ 先前生成的实现步骤 $\text{Generate}(\cdot)$ 是集成示例和用户查询以生成实现步骤的函数。

6.3.3 生成扩展 API 描述。然而，LLM 有时可能会生成包含复合功能的 API 描述，需要多个 API 才能完全实现。为了解决这个问题，我们进一步指导 LLM 扩展潜在 API 集，确保更广泛的覆盖范围并降低

6.4 上下文集成代码生成

6.4.1 编码和检索。一旦生成了潜在的应用编程接口描述，它们就会使用存储库中用于编码应用编程接

6.4.2 生成目标代码。最后，在输入 LLM 之前，所有检索到的 API 都

7.2 RQ5: AllianceCoder 在 API 预

RQ6: 与使用代码片段相比，为 API 检索生成自然语言描述

7.1 RQ4: AllianceCoder 在

表 4 展示了不同 LLM 中不同方法的性能比较。作为基线，我们包括了初步研究中的 ContextLLM。值得注意的是，AllianceCoder 在 Pass@1 指标方面表现出实质性的改进，增加了大约 10-20%。此外，与 CoderEval 数据集上的

ContextLLM 和 ConAPIGPT 相

比，AllianceCoder 的性能提升尤其明显。这一结果强调了

AllianceCoder 中 API 检索的有效性。然而，需要注意的是，AllianceCoder 在双子座 CoderEval

条件下，Pass@3 和 Pass@5 的最大可实现改进保持在 5% 左右。我

总结 1: AllianceCoder 在所有基线上都实现了 SOTA 性能，在 Pass@1 方面尤其显著。在带

7.2 RQ5: AllianceCoder 在 API

预测方面的表现如何。其预测与理为了评估 AllianceCoder 在 API 预测中的有效性，我们首先检查

AllianceCoder 获得的 API 召回数，如表 6 所示。在该表中，High 表示

AllianceCoder 召回的 API 数量多于目标函数实际所需数量的测试用例

表 7 显示了 AllianceCoder 和 AllianceCoder 与上下文组合的 API

召回率。在此表中，Recall 表示 AllianceCoder 在所有测试用例中的

API 召回率，BRecall 表示 AllianceCoder 和 ConAPILLM 都成功通过的测试用例的 API 召回率

(即 API 检索的理想条件)，BRecall 表示 AllianceCoder 的 API

总结 2: AllianceCoder 倾向于预测比实际需要的更多的 API，

并为 LLM 提供潜在调用 API

7.3 RQ6: 与使用代码片段相

比，为 API 检索生成自然语言描述。尽管大多数现有的预训练模型试图在

上下文信息中的 API，API 召回率仍然相对较低。尽管如此，AllianceCoder 的整体性能接近

表 4：不同方法与各种 LLM 的性能比较（以粗体显示的最佳性能）。

Approach	RepoExec			CoderEval		
	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
RepoCoder _{GPT}	19.72	23.10	25.91	14.35	17.39	19.13
RLCoder _{GPT}	31.55	36.62	38.87	16.52	19.13	21.30
Context _{GPT}	29.01	34.08	38.59	30.43	36.52	37.83
AllianceCoder _{GPT}	34.93 (10.7%↑)	40.28 (10.0%↑)	41.97 (8.0%↑)	36.52 (20.0%↑)	40.00 (9.5%↑)	41.30 (9.2%↑)
ConAPI _{GPT}	37.75	44.51	47.04	36.52	41.30	41.74
RepoCoder _{Gemini}	25.63	31.26	32.95	13.91	15.65	16.09
RLCoder _{Gemini}	12.68	15.21	18.31	12.17	12.61	13.04
Context _{Gemini}	30.99	34.08	35.77	23.04	26.09	27.39
AllianceCoder _{Gemini}	35.49 (14.5%↑)	39.16 (14.9%↑)	41.13 (15.0%↑)	24.78 (7.6%↑)	26.52 (1.6%↑)	27.82 (1.6%↑)
ConAPI _{Gemini}	38.59	41.69	44.51	25.22	27.39	28.70

表 5：文本到文本和文本到代码检索之间的性能比较。

Approach	RepoExec			CoderEval		
	Pass@1	Pass@3	Pass@5	Pass@1	Pass@3	Pass@5
AllianceCoder _{GPT}	34.93	40.28	41.97	36.52	40.00	41.30
AllianceCoder(Code) _{GPT}	33.71 (3.5%↓)	39.38 (2.2%↓)	41.93 (0.1%↓)	23.47 (35.7%↓)	25.65 (35.9%↓)	26.96 (34.7%↓)
AllianceCoder _{Gemini}	35.49	39.16	41.13	24.78	26.52	27.82
AllianceCoder(Code) _{Gemini}	28.45 (19.8%↓)	33.80 (13.7%↓)	34.65 (15.8%↓)	18.26 (26.3%↓)	20.43 (23.0%↓)	21.30 (23.4%↓)

表 6：API 召回计数的比较：

AllianceCoder 与实际 API 计数。

Approach	RepoExec			CoderEval		
	Higher	Equal	Lower	Higher	Equal	Lower
AllianceCoder _{GPT}	74.45	6.9	18.61	72.64	4.7	22.64
AllianceCoder _{Gemini}	51.42	13.25	35.33	58.49	9.9	31.60

表 7：AllianceCoder 和

AllianceCoder 与上下文组合的

API 召回率比较。

Approach	RepoExec			CoderEval		
	Recall	BRcall	CRcall	Recall	BRcall	CRcall
AllianceCoder _{GPT}	20.38	21.33	29.23	16.81	24.00	46.22
AllianceCoder _{Gemini}	14.62	16.05	23.46	10.08	15.38	39.50

在无监督的预训练阶段，这两种模式之间仍然存在持续的对齐差距。这种不对齐导致代码 - 查询对的表为了研究这种语义差距对整体性能的影响，我们进行了一项实验，其先来自存储库的 API 直接编码为表示向量，然后基于这些向量进行检索。表 5 总结了结果，产生了几个关键观察结果。

首先，性能对存储库中代码的具体实现细节高度敏感。值得注意的是，对于两个 LLM，CoderEval 数此外，LLM 生成准确且语义上有意义的描述的能力在检索性能中发挥着至关重要的作用。不同的 LLM 在同一数据集中表现出不同程度的性能下降。例如，在 RepoExec 中，总结 3：直接使用代码进行检索会导致 AllianceCoder 的下降低的整体性能。这种性能下降的程度取决于 LLM 和数据集的训练数据。8 相关作品 8.1 代码生成，它们的输出格式存储库级代码生成的任务对于现实世界场景中的智能软件开发越来越受到关注 [33,44,48,56]。传统的代码生成方法可以

主要分为基于规则的方法 [22]、基于统计概率模型的方法 [41] 和基于深度学习的方法 [5、9、23、48]。随着大语言模型 [1、17、例如，ReCode [21] 通过子树检索改进了神经代码生成。RedCoder [38] 提出首先检索给定代码功能描述的 top-k 候选代码，然后聚合这些候选代码，最终生成目标代码。DocPrompting [61] 采用代码留档来改进代码生成，解决了生成未知函数和库代码的挑战。ReACC

8.2 存储库级代码生成 类似代码存储库级代码生成利用了跨代码存储库可用的广泛上下文，已成为该领域的中心研究重点。研究试图改进存储库级代码生成 [13,15,35,39,49,57]。RepoCoder [57] 和 De-Hallucinator [15] 采用迭代检索和生成框架。RepoMinCoder [28] 建立在传统的检索增强生成方法的基础上，引入了额外的一轮基于信息丢失的筛选和排名。R2C2-Coder [11] 首先构建一个候选检索池，然后从池中检索每个完成位置的相关内容，将其组装成完成提示。上述方法提取目标代码以进行检索。然而，它们忽略了代码内部的内在结构。为了解决这个问题，CoCoMIC [13] 和 RepoHyper [39] 构建方法级图来增强检索过程。尽管如此，这些方法未能捕获语句级结

由强化学习驱动的性别划分方法。RepoGenReflex [46] 是另一个由强化学习驱动的框架。CoCoGen [6] 在 KNN-LM [24]、KNM-LM [45] 和 FT2Ra [19] 等方法中，为每个生成的令牌触发检索过程。因此，随着生成的序列长度的增加，导致检索时间的显著增加。CodeAgent [58] 和 ToolGen [47] 研究了工具调用机制的集成。此外，还引入了 RepoEval [57]、RepoBench [34] 和总之，这些方法被证明对存储库级别的代码生成是有效的。然而，确定哪些 API 可以在存储库中调用仍然是一个具有挑战性的问题。为了解决这个问题，我们首先为存储库中的每个 API 生成描述符级别的有效性威胁。我们首先考虑内部威胁。第一个内部威胁与我们实验数据集的范围有关。当我们评估我们在 Python 编程语言上的方法时，这可能限制我们的发现。第二个内部威胁源于 LLM 的选择。由于计算限制，我们只评估了两个 LLM（GPT 和双子座变体）。第三个内部威胁涉及我们的评估指标。虽然 Pass@k 有效地衡量功能正确性，但它不评估代码质量。外部威胁。第一个外部威胁是潜在的数据泄露，尽管我们的数据集在生成前经过设计导致模型失忆。工作可以结合静态分析或人工评估来解决这一威胁。

[54] 田宇, 张少雷, 杨峰。2024. AutoRag: 大型语言模型的自主检索增强生成。arXiv 预印本 arxiv: 2411.19443 (2024)。

[55] 道光赞, 北辰, 永顺公, 曹君

[56] 赞道光、陈备、杨德建、林泽

[57] 张凤姬, 陈贝, 张悦, 姜洋,

[58] 张克驰, 李佳, 李戈, 施贤

[59] 张文瑞, 傅铁航, 袁婷, 张

[60] 周淑燕、Uri Alon、Sumit

[61] 周淑燕、Uri Alon、Frank F

[62] 朱玉琪, 李佳, 李戈, 赵云

2007年2月20日收到; 2009年

3月12日修订; 2009年6月5

日接受。在 AAAI 人工智能会议

论文集, 卷。38。437-445。